Writeable Signals



- Creating a signal

```
readonly firstSignal = signal(42);
```

- Reading the value

```
let val = this.firstSignal();
```

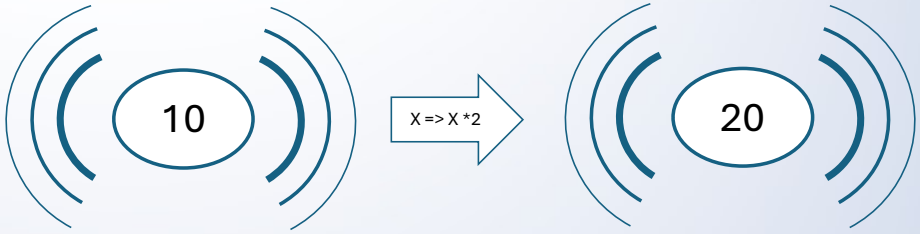
- Binding to the value

```
<h1>
    First Signal: {{firstSignal()}}
</h1>
```

- Modifying the value

```
this.firstSignal.set(10);
this.firstSignal.update(value => value + 1);
```

Computed Signals



- Creating a computed signal

```
readonly derived = computed(() => firstSignal() * 2);
```

- Reading the value

```
let val = this.derived(); // same as writeable
```

- Binding to the value

```
<h1>
    Derived Signal: {{derived()}} // same as writeable
</h1>
```

- ~~Modifying the value~~

Computed signals

- It's ok to
  - Use more than one signal

```
computed(() => x() * y())
```

  - Use writeable or computed signals

```
computed(() => x() * derived())
```

  - Use constants and immutable non-signal data

```
computed(() => x() * Math.PI)
```

- But don't
  - Use asynchronous code

```
computed(async () => x() * await calc())
```

  - Use changeable data that is not signal

```
computed(() => x() * Date.now())
```

  - Cause side effects

```
computed(() => x() * j++)
```

  - Modify or create other signals

```
computed(() => {
    x.update(v => v + 1);
    return x() * 10;
})
```